



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Availability and Reliability

Michele Colajanni

Dipartimento di Informatica – Scienza e Ingegneria
michele.colajanni@unibo.it

Mission critical and Safety critical

Regardless of business or industry sector, every successful operation is dependent on multiple interacting systems often including computer, electronic, and mechanical systems. While each component has an important role to play, some applications, processes, and subsystems are more fundamental than others

- ***A mission critical system is a system that is essential to the survival of a business or organization.*** When a mission critical system fails or is interrupted, business operations are significantly impacted
- ***A safety critical system is a system that, if it fails, may result in serious environmental damage, injury, or loss of life***



Examples

- **Safety critical systems** include control systems for aircraft, for nuclear plants, electricity grid systems, and emergency communications systems
- For most IT businesses and organizations, database systems and process control servers are **mission critical systems**
 - Data centers and database servers for Internet applications must be protected from scenarios resulting in the potential loss of all functions, such as power shortages or faulty hardware
 - Most system backups are set up in a redundant and seamless manner



Unavailability of IT services



GOV.UK

Service name

Sorry, the service is unavailable

You will be able to use the service on Monday
19 November 2018.

We have not saved your answers. When the service is
available, you will have to start again.

[Contact the Tax Credits Helpline](#) if you need to make
changes to your claim or speak to someone about your tax
credits.



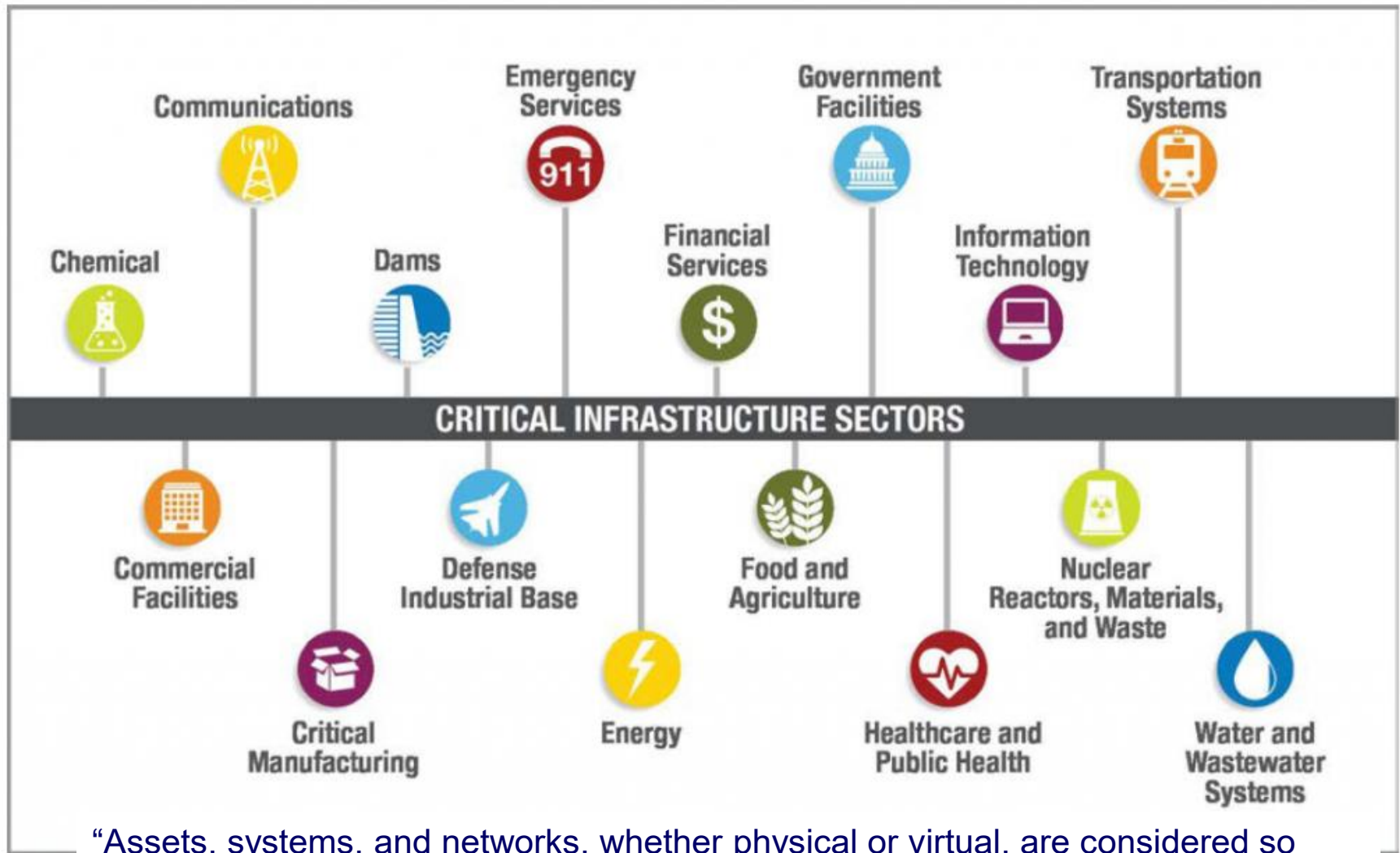
Mission critical systems

A system is considered *mission critical* when any of the following conditions are met:

1. Critical business functions and applications are affected
2. Information is compromised (e.g., loss of data or damage of data integrity)
3. Affected parties are subject to legal, regulatory, or financial costs
4. Reputation is adversely affected in a significant manner



Critical infrastructures



“Assets, systems, and networks, whether physical or virtual, are considered so vital to the United States that their incapacitation or destruction would have a debilitating effect on security, national economic security, national public health or safety, or any combination thereof” - *Presidential Policy Directive 21 (PPD-21)*



Many principles come from *nuclear engineering* sector



Technologies and procedures are reinforced by major incidents:
Three Mile Island (1979), Chernobyl (1986), Fukushima (2011)



Three Mile Island, Pennsylvania (March 28, 1979)

- The most serious nuclear accident in US history took place at a brand-new facility that was lauded for its state-of-the-art design, efficiency and affordability during an era of energy crises
- **It began when a pressure valve in one of the reactors failed to close**, allowing cooling water—contaminated with radiation—to drain into some buildings
- **Control room operators made critical errors as they struggled to contain the crisis.** The core had heated to over 2,200 degrees Celsius, just 530 degrees below a disastrous meltdown

“How the Three Mile Island accident was made even worse by a chaotic response” (<https://www.history.com/news/three-mile-island-evacuation-orders-controversy>)



The other two major incidents

Chernobyl (April 26, 1986) - *A failed stress testing experiment*

- A mismanaged experiment at one of the facility's four reactors created a sudden power surge, which in turn led to a series of blasts that blew the 1,000-ton steel top off of the reactor

Fukushima (March 11, 2011) – *An unexpected black swan*

- A massive undersea earthquake—the largest ever recorded in Japan—occurred off the northeast coast of the country's Tōhoku region. The **9.1 magnitude quake** triggered a tsunami which struck the Fukushima Daiichi nuclear plant. During the emergency, each of the three operational nuclear reactors at the plant shut down successfully, but **the backup power and cooling systems failed**



Aviation industry is another source of key information for safety critical systems



Nevertheless, we experience several aviation incidents



When competition may kill people: “The 737 MAX jet cases”

At the basis: Competition Airbus vs Boeing

- In October 2018, Lion Air Flight 610 crashed just minutes after taking off from Jakarta, Indonesia, killing 189 people
- In March 2019, another Max, Ethiopian Airlines Flight 302, crashed minutes after take-off; all 157 people on board died
- It is a long and complex story, but it is likely that the **new plane's automated software systems**, no **specific training** (and consequent disabling pilots) may have played a big role
- Short time-to-market (*time!*) and no new pilot training (*costs!*) are important parts of the cause. No listening to Boeing engineers because of time deadlines has again a cos-related motivation



The Boeing 737 MAX case

- Investigation about the two crashes (the Air industry is one of the most serious and severe) evidenced the role of the control software system (**MCAS**) that was supposed to keep people safe, but instead contributed to tragic deaths when triggered by a single faulty sensor
- One investigation video about this story (others on YouTube):
<https://www.pbs.org/wgbh/frontline/film/boeings-fatal-flaw/>
- This film explains how **intense market pressure** and **failed oversight** contributed to a catastrophic crisis for Boeing, one of the world's most iconic industrial names. Looking backwards, you can see the trajectory to tragedy along the way at Boeing: "If MCAS hadn't been on those planes, those planes wouldn't have crashed."
- Boeing agreed to pay a **\$2.5 billion** settlement: \$500 million to the families of the victims and most of the rest to compensate the airlines



Main principles

- **Availability**
- **Reliability**
- **Resiliency**

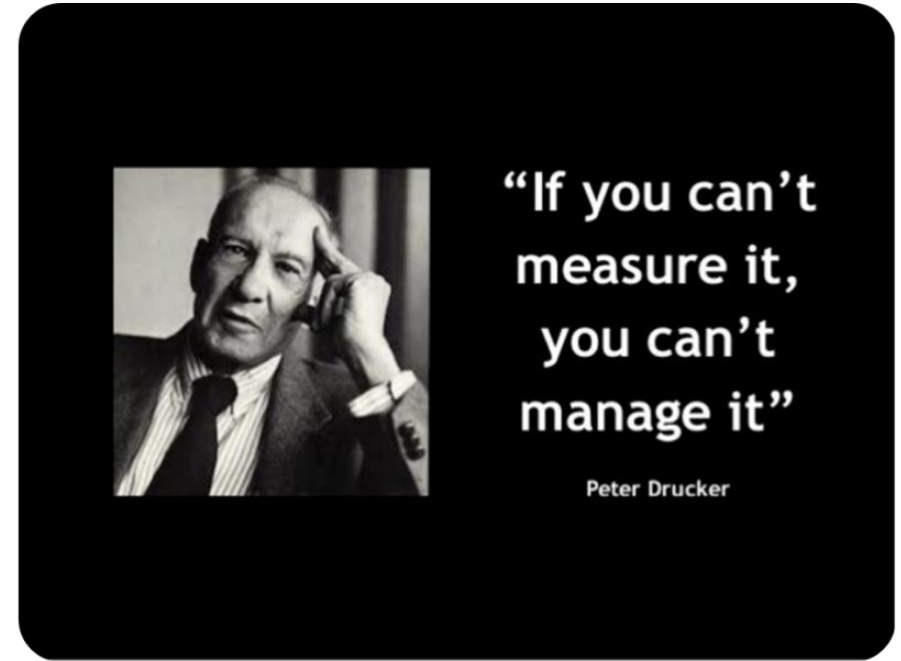
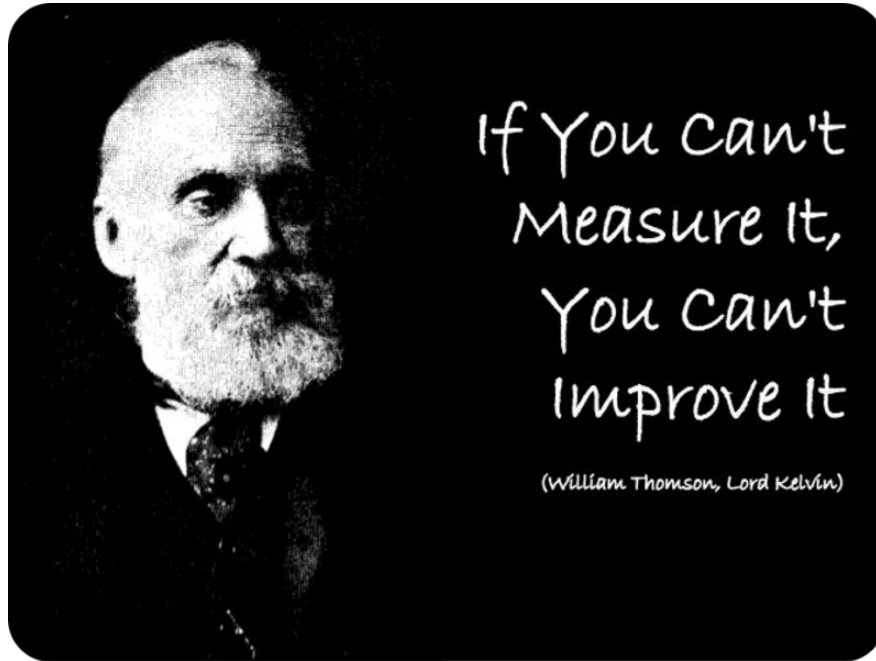
Describe the amount of time when a service runs in a *healthy state* without significant downtime

Describes how quickly an application *recovers from failure*



Measurement

Scalability, Availability and Reliability start from *measures*



"If you do not know the system and the problem you want to manage, you are wasting your time measuring"

"If you are not committed to do what it takes to improve a metric, you might not bother measuring it at all"



Complicated and complex problems

You and your top management must have clear ideas and commitment about the system you are dealing with

- Any **complicated problem**, if approached properly, is really a collection of smaller problems and (linear) interactions. → They can be analyzed, controlled and solved through traditional engineering approaches
- A **complex problem** results from networks of multiple interacting causes that cannot be individually distinguished. *Emergent patterns* (small inputs may result in disproportionate effects) can arise from dynamic interactions between the non-linearly connected parts. → These problems cannot be solved once and forever, but require to be systematically managed because any intervention merges into new problems as a result of the interventions dealing with them: an iterative and continuously adaptive approach must be used



Availability



Availability

Availability refers to the percentage of time that the system, infrastructure or service remains operational (**uptime**) in order to serve its intended purpose during the entire time interval of observation (**total elapsed time**)

Availability =

$$\frac{\text{Total elapsed time} - \text{Downtime}}{\text{Total elapsed time}}$$

$$\frac{\text{Uptime}}{\text{Uptime} + \text{Downtime}}$$

Example

For a cloud-based service, that must operate h24, availability relates to the time that the data center is accessible and delivers the intended service as a proportion of the entire duration for which the service is purchased



Availability is typically measured on a '9' scale

Availability %	Downtime per year	Downtime per month	Downtime per week
90% (1 nine)	36.5 days	72 hours	16.8 hours
99% (2 nines)	3.65 days	7.20 hours	1.68 hours
99.5%	1.83 days	3.60 hours	50.4 minutes
99.9% (3 nines)	8.76 hours	43.8 minutes	10.1 minutes
99.95%	4.38 hours	21.56 minutes	5.04 minutes
99.99% (4 nines)	52.56 minutes	4.32 minutes	1.01 minutes
99.999% (5 nines)	5.26 minutes	25.9 seconds	6.05 seconds
99.9999% (6 nines)	31.5 seconds	2.59 seconds	0.605 seconds
99.99999% (7 nines)	3.15 seconds	0.259 seconds	0.0605 seconds

Mission critical services

The famous five nines



What percentage is acceptable?

90%

95%

99%

99.9%

99.99%

99.999%

99.9999%

Easy!

The answer depends on (how much critical is) the service



System view

Availability is also the probability that at time t the system works

Availability = **99%** (two nines)

→ Downtime per year = $0,01 \cdot 365.2425$ d
= 3d 15h 39m 29.5s

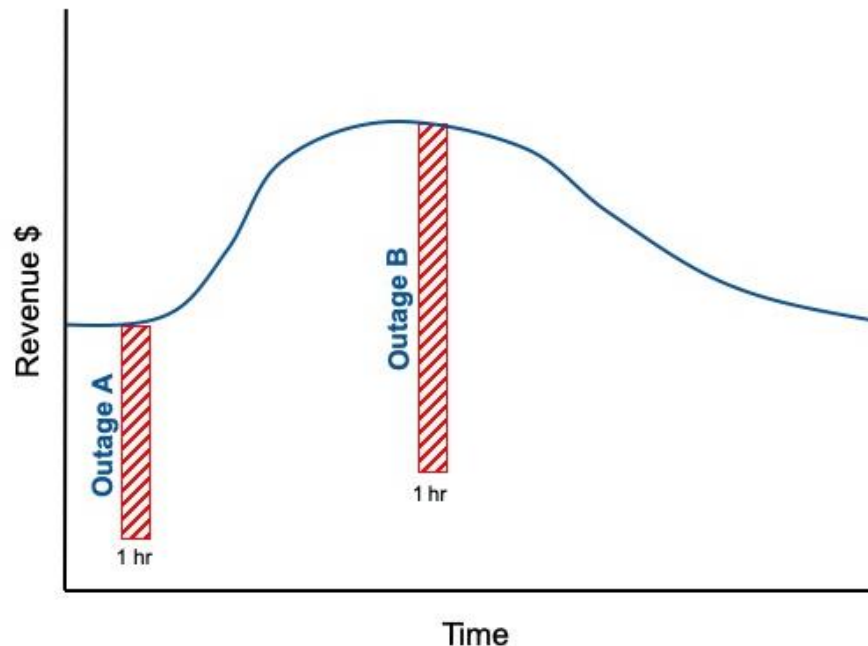
The system engineers of a provider typically evaluate the **quality of a service** in terms of *availability* that is measured as a percentage of the time that the service remains operational



Different times of outage, different impacts

However, units of time are not equal in terms of business impact: an interruption during the busiest part of the day would be much worse than an issue for late at night or early in the morning

- Indeed, this “slow” period is often known as ***maintenance windows*** during which many companies schedule maintenance operations (e.g., patching, hardware replacement) because the impact of inoperability or limited operation is much smaller



Reliability



Reliability vs. Availability

Reliability refers to the probability that the system will meet certain performance standards in yielding correct output for a desired time duration

This may seem identical to how we define **Availability**, but there are some key differences

- 1) **Availability is a *system-oriented metrics***: it looks at whether the service is working, independently whether a user is accessing it or not
- 2) If users accessed the service uniformly across all features and at all times, Availability would be equal to Reliability. But this is generally never the case



Reliability

- Reliability is a **customer-oriented metrics**: it considers how a user perceives the *quality of a service*
- Reliability accounts for **workload** and **actual usage**. It converts availability (a *system metrics*) into a *customer measure*
- It can be also defined as the likelihood of a service functioning as expected when accessed by a user
 - Hence, mathematically, the Reliability is a subset of the Availability
- Reliability (and not Availability) should be considered as a basis for **Service Level Agreement (SLA)** by a customer
(Not surprisingly, a service provider prefers *availability*, while customers should strive to get contracts based on *reliability*)



Example: *Which is the most reliable service?*

	Service A	Service B
User log-on page	97% availability	99% availability
Catalog search	97% availability	96% availability
Site settings page	97% availability	90% availability

- Service A would have the best availability and the best reliability if the workload was uniform
- On the other hand, Service B is better if
 - the log-on page is used by 100% of users
 - the catalog search is used by 90% of users,
 - the site settings page is used by only 30% of users

Reliability accounts for **workload** and actual usage



Service Level Agreement (SLA)

A service-level agreement (SLA) is a contract between a service provider and its customers that documents what services the provider will furnish and defines the service standards the provider is obligated to meet

- When IT outsourcing emerged in the late 1980s, SLAs evolved as a mechanism to govern such relationships
- SLAs set the expectations for a service provider's performance and established penalties for missing the targets (and, in some cases, bonuses for exceeding them)
- The SLA should include not only a description of the services to be provided and their expected service levels, but also metrics by which the services are measured, the duties and responsibilities of each party, the remedies or penalties for breach, and a protocol for adding and removing metrics



Topics to be considered in SLAs (1/2)

1. **Service level requirements:** the key deliverables and scope that customers agree to receive from the service provider. This section should cover what will be provided, how it will be provided, reliability, and responsiveness
2. **General obligations.** As an SLA is a two-way agreement between parties, each has a certain level of responsibility. In most cases, the service provider assumes the vast majority of the obligations in this agreement. However, in some cases, clients may also be required to fulfill some duties, such as providing payment or holding review sessions within a specific timeframe
3. **Response requirements.** In the event of service interruptions, providers will need to outline the type of action that will be taken. SLAs should go into as much detail as necessary, defining such parameters as the categories of incidents and how providers will respond to each type of event
4. **Metrics** depend on the type of service covered in the SLA. An IT provider should add metrics such as service availability, quality of service and average incident response times metrics related to cybersecurity
5. **Costs** will vary depending on the type and quality of service covered in the SLA



Topics to be considered in SLAs (2/2)

6. **Monitoring and reporting requirements.** SLAs should describe how service providers collect data, how often that information will be collected, and how and when that information will be communicated to the customer
7. **Business outcomes.** Some customers are concerned about business outcomes in SLAs. Rather than emphasizing service availability, an SLA may promise specific results, such as the ability to perform specific actions within a software
8. **An indemnification clause** transfers risk—often a legal liability—from one party to another. In the case of an SLA, an indemnification clause may require the service provider to pay the customer if the contract is breached. For instance, if the provider breaches the contract and a third party sues the client, the indemnification clause would require the service provider to cover the legal costs
9. **Penalties** are incurred when either party fails to meet its end of the obligation. They can include financial penalties, such as monetary compensation, service credits, or even contract termination
10. **A termination clause** outlines the requirements for ending the service agreement. They should focus primarily on points such as timelines, costs, service continuity, and potential penalties



Service Level Commitment (SLC)

- Many (big) service providers define a master service agreement to establish the general terms and conditions in which they will work with customers
- An SLA is bidirectional contract and involves two teams
- A **Service Level Commitment (SLC)** is a one-directional obligation that establishes what a provider can guarantee to its customers at any given time



Reliability

- The measurement of **availability** is driven by **time**
- The measurement of Reliability is driven by the usage and by the **frequency** and **impact of** (intentional or unintentional) **failures** (including cyber attacks) and not only by **time** as for Availability

Reliability can be used also to understand how well the service will be available in different conditions

- For instance, a service may be available with an SLA commitment of 99.999%, but vulnerabilities to cyber attacks may cause outages beyond the control of the provider
- As a result, with no **maintenance** and **no incident management** procedures, the service may be compromised for a long time, thereby reducing the effective availability of the service



Metrics related to reliability

There are several ways to measure the probability of failure of system components that impact the reliability of the system.

Common metrics:

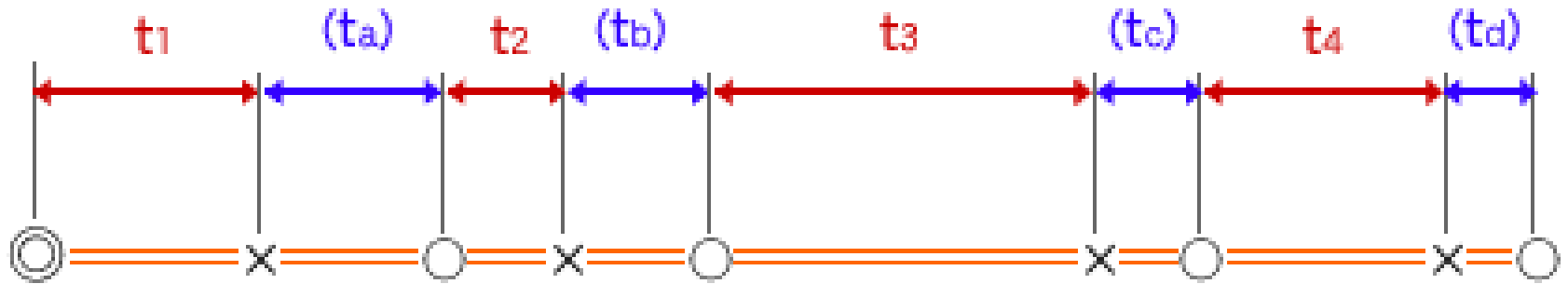
- The **Mean Time Between Failures (MTBF)** represents the time duration between a failure of a component that can be fixed. It reflects how long equipment can operate without being disrupted
 - If the system/component is non-repairable, the **Mean Time To failure (MTTF)** is a preferred statistic to MTBF
- The **Mean Time To Repair/Recover (MTTR)** represents the time duration to detect and repair a failed system component

$$\text{MTBF} = \frac{\text{Total elapsed time} - \text{Downtime}}{\text{Number of failures}}$$

$$\text{MTTR} = \text{Downtime} / \text{Number of repairs}$$



Example

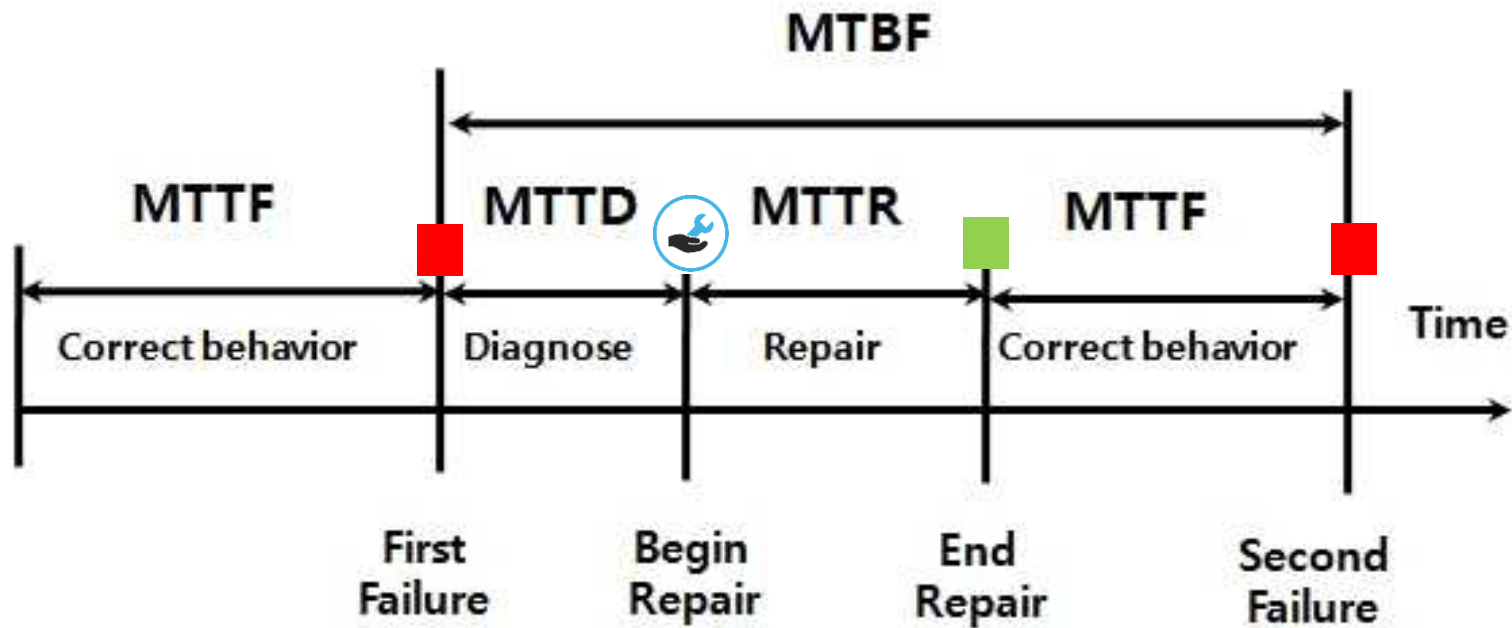


$$MTBF = \frac{\sum_{n=1}^n t_n}{n} = \frac{t_1 + t_2 + t_3 + t_4}{4}$$

$$MTTR = \frac{\sum_{n=a}^n t_n}{n} = \frac{t_a + t_b + t_c + t_d}{4}$$



The overall picture



Mean Time to R...

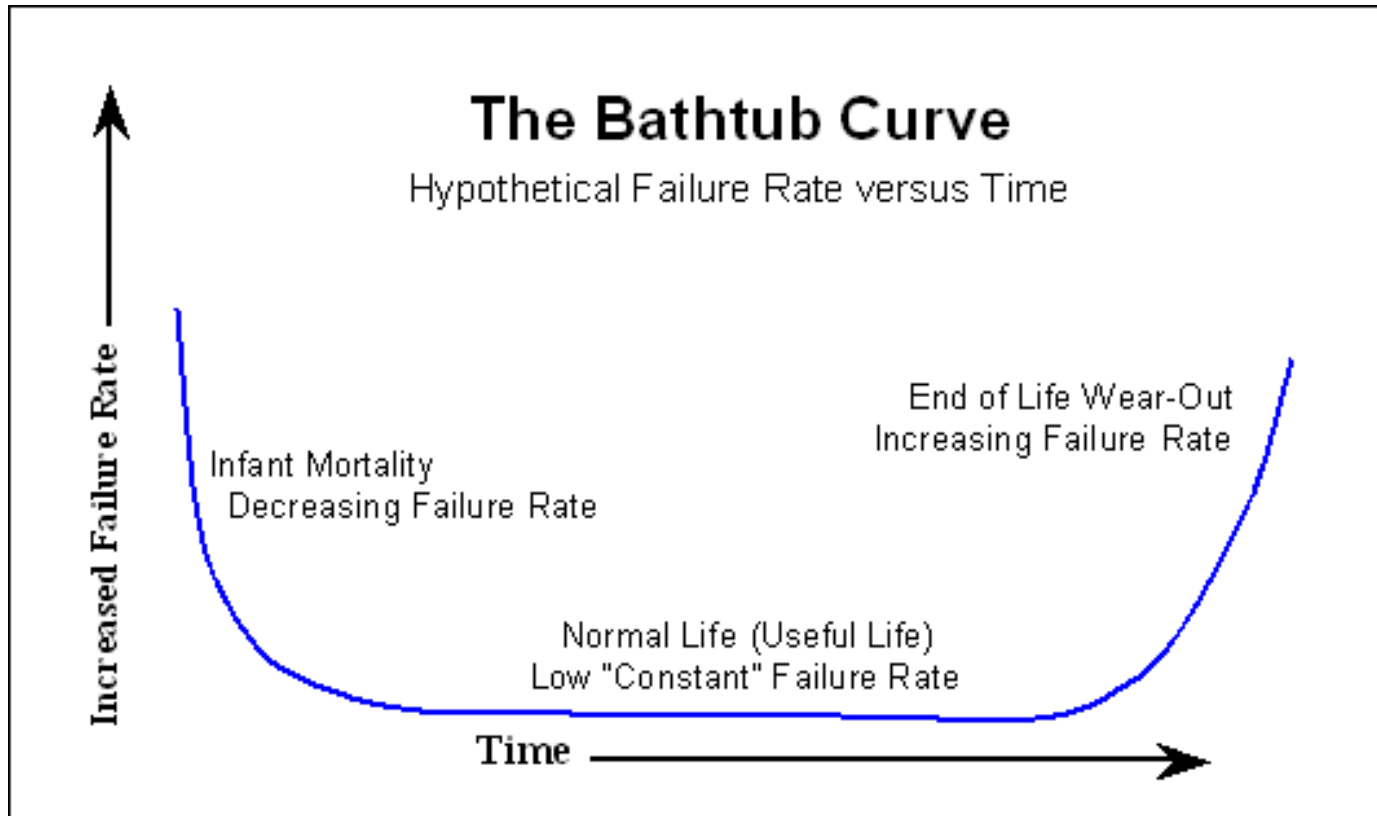
It is unavoidable for technology to fail in different ways

- **Mean Time to Repair** is the average time to repair and return a component or system to working (typical technological view)
- **Mean Time to Recover** is a key indicator of an organization's ability to maintain its systems, equipment, applications, and infrastructure, as well as its efficiency in repairing such equipment in the event of an outage (still, basically a technological view)
- **Mean Time To Resolve** is the average time to completely resolve an incident, including recognizing the problem, fixing the effects, and taking steps to prevent the event from happening again

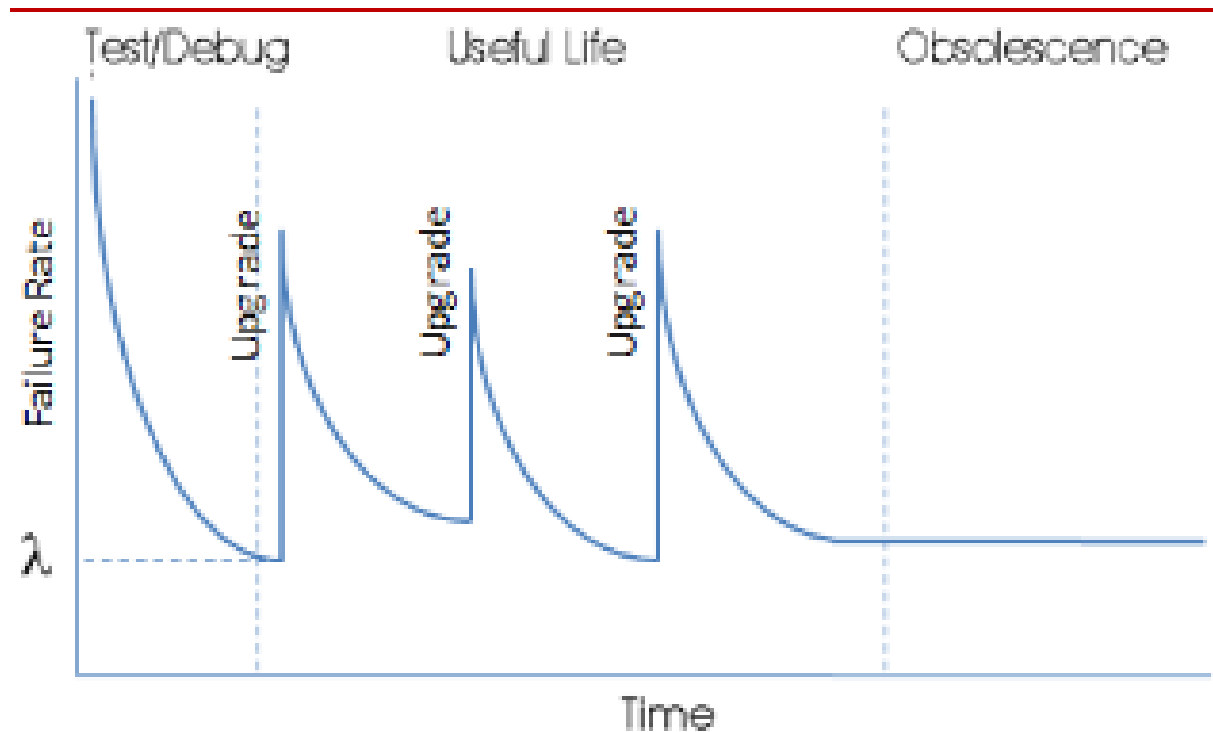
Understanding MTTR gives businesses a sense of how fast and efficiently a (IT) department can respond to breakdowns and resume normal operations: any low MTTR rate indicates a valid department



The model for typical hardware components



The model for typical software components



Business view

- The business does not communicate in terms of availability nor of reliability but in terms of **business metrics**, such as
 - Revenue
 - Cost
 - Margin
 - Return on Investment
- The main difference is that all these terms are measured in **money**, not as a clock time
- Using the uptime figure for a service as an availability measure is inaccurate because it does not capture blackout, software bugs, cyber attacks, network interruptions or any other issue rendering service inoperative despite the server uptime status



Be aligned with desired business outcomes

- 1) What are the main goals of the business?
 - 2) What is the *customer value* proposition?
- **Choose metrics** that comprehensively measure the reliability of your product or service, e.g.,
 - the ability of a customer to buy a product from your e-commerce site (login, search, add to cart, and check out)
 - the proportion of file downloads successfully completed in less than 4 seconds
 - the success rate of posting a message to a social media platform and the ability of others to view it
 - **Measure metrics** that are aligned with the desired outcomes and:
 - keep the big picture at the forefront
 - help business colleagues understand how the technology team contributes to value creation



Some business metrics (*examples*)

Transactional metrics aligned to the desired business outcome are better choices

- **Rates:** Log transactional rates such as logins, add to cart, registration, downloads, orders, etc. It is possible to apply statistics or other analytical methods to establish thresholds indicating an unusual deviation in the transaction rate
- **Ratios:** The proportion of undesired or failed outcomes, such as failed logins, abandoned shopping carts, and HTTP 400 (bad requests), can be useful for measuring the quality of service. Analysis of such ratios can establish unusual deviation levels from the desired business goals
- **Patterns:** Normal transaction patterns can identify expected activity, such as order rates increasing when an item is first available for sale or download rates increasing in response to a viral social media video. The absence of an expected pattern change can signal an availability issue with the provided product or service



Lesson learned

- 1) **Not measuring availability** is a bad approach to service management
- 2) **Measuring availability in clock time** does not give the real measure of quality-of-service impact on business
- 3) **Measuring reliability** instead of availability is a better choice because it includes the time duration between a component failure of the system (*Mean Time Between Failures* - MTBF) and the time to repair a failed system component (*Mean Time To Repair* - MTTR). However, it still gives a *system operational focus* that business might not understand nor appreciate
- 4) **Measuring availability in terms of transactional metrics** that are tied to the desired business outcome is definitively the best approach

